

Dispensa · Classe 4 · Informatica (Laboratorio Professionale) · v1.0 · 29/09/2026

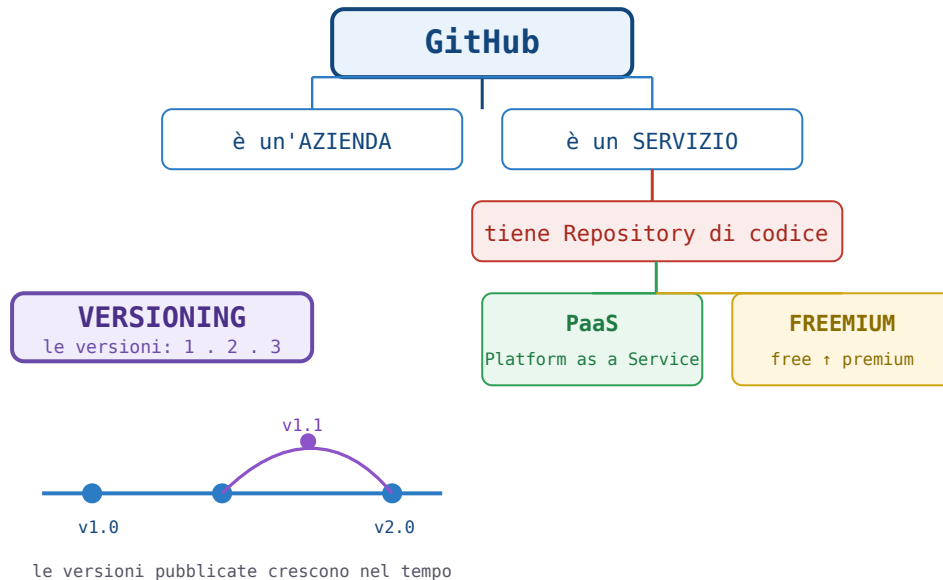
Git, versioning e Intelligenza Artificiale

Git e GitHub, le versioni, e come usare Gemini e NotebookLM

Un unico percorso: come teniamo in ordine il lavoro con **Git e GitHub** e il **versioning**, con tanti esempi (i grafi dei casi d'uso), e poi come usare l'**Intelligenza Artificiale** — **Gemini** con un file **.md** e **NotebookLM** con un **PDF**. Senza comandi da digitare, tutto da browser. Questo materiale esiste anche in formato **.md** da dare a Gemini.

1. Partiamo dalla nostra lavagna

Ecco, ridisegnato in ordine, lo schema che abbiamo costruito insieme alla lavagna. Ogni riquadro è una tappa: nelle prossime pagine le apriamo una per una.



Lo schema della lezione: GitHub è azienda e servizio; tiene repository di codice; come servizio è una piattaforma (PaaS) col modello Freemium; e il progetto cresce per versioni (versioning).

[Qui andrà la foto della VOSTRA lavagna, scattata durante la lezione]

La lavagna della nostra lezione (da inserire dopo la spiegazione).

2. GitHub: un'azienda e un servizio

Alla lavagna abbiamo scritto due parole sopra "GitHub": **azienda** e **servizio**. Vediamo perché tutte e due sono vere.

È un'azienda

Dietro GitHub c'è una **società vera** — **GitHub, Inc.**, nata nel 2008 e dal 2018 di proprietà di **Microsoft**. Come ogni azienda deve mantenersi: ci arriviamo al punto "Freemium".

È un servizio (online)

GitHub è un **sito** dove metti i tuoi progetti **online**: per tenerli al sicuro, mostrarli e lavorarci in gruppo. Non è un programma che installi col CD: lo usi dal **browser**.

Attenzione a non confondere: **Git** è lo **strumento** (la tecnologia che tiene la storia di un progetto, gratis e di nessuno); **GitHub** è il **servizio online** (l'azienda) dove quei progetti vivono in rete. Git è il **motore**, GitHub è il **garage online**.

3. Che cosa tiene: i repository di codice

Sulla lavagna, sotto "servizio", abbiamo scritto "**repository di codice**": è ciò che GitHub custodisce.

Un **repository** (in breve "repo") è la **cartella-progetto** con dentro i file **e tutta la loro storia**. Ogni salvataggio della storia si chiama **commit** (una "fotografia" del progetto con un messaggio). Se lavori in gruppo, puoi aprire un **branch** (una strada di lavoro parallela) e poi riunirlo con un **merge**.

Esempio. Il tuo gioco "SpaceRun" è un repository su GitHub: dentro c'è il gioco **e** il diario di tutte le modifiche. Puoi tornare a com'era la settimana scorsa in qualsiasi momento.

4. Che tipo di servizio è: PaaS

Alla lavagna: **PaaS = Platform as a Service**, cioè "piattaforma **come servizio**". Vuol dire che qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza costruirti e mantenere i computer, i server e tutto il resto.

Analogia della cucina. Per aprire un ristorante puoi **costruirti la cucina da zero** (tanta fatica) oppure **affittare una cucina già attrezzata** e pensare solo a cucinare. GitHub è la cucina già pronta: tu porti il progetto, il resto lo mette lui.

La famiglia "...aaS"

PaaS non è da solo: fa parte di una famiglia di servizi online, tutti con la sigla che finisce in "**aaS**" (as a Service). Cambia quanto lavoro fai tu:

Sigla	Nome	Cosa ti danno pronto	Esempio
IaaS	Infrastructure as a Service	Solo i "muri e la corrente": server nudi.	Amazon AWS, Google Cloud
PaaS	Platform as a Service	La cucina già attrezzata: piattaforma pronta.	GitHub, Netlify
SaaS	Software as a Service	Il piatto già servito: un programma pronto nel browser.	Gmail, Google Documenti

Come ricordarli — la pizzeria. **IaaS** = affitti il locale vuoto. **PaaS** = affitti la cucina attrezzata. **SaaS** = ti fai consegnare la pizza pronta. Più scendi, meno fatica fai.

5. Come si paga: il modello Freemium

Sulla lavagna: **Freemium = FREE + PREMIUM**. È una parola nata unendo **free** (gratis) e **premium** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti; se vuoi funzioni **in più**, paghi un abbonamento.

Esempio che conosci. Tanti giochi per telefono e app come Spotify funzionano così: gratis con qualche limite, a pagamento per togliere pubblicità o avere funzioni extra. **GitHub è uguale:** a te studente basta e avanza la parte gratis.

Ecco perché "azienda" e "Freemium" stanno insieme: GitHub è un'azienda, quindi deve guadagnare; lo fa lasciando gratis l'uso normale e facendo pagare le funzioni avanzate a chi ne ha bisogno (aziende, team grandi).

6. Il versioning: le versioni 1.2.3

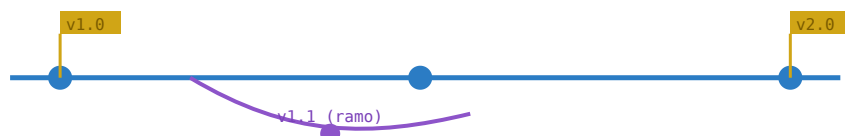
L'ultima parte della lavagna: **VERSIONING**, con il numero **1.2.3**. Quando un progetto raggiunge un punto "buono", gli si dà un **numero di versione** fatto di tre cifre:

Cifra	Nome	Aumenta quando...
1	major	cambi tutto , in modo grosso (magari incompatibile col vecchio)
2	minor	aggiungi una funzione nuova , ma il resto funziona ancora
3	patch	correggi solo un bug , niente di nuovo

Esempio. v1.0.0 prima versione giocabile → aggiungi i suoni: v1.1.0 → correggi un bug: v1.1.1 → rifai tutto da capo: v2.0.0.

Il grafo che abbiamo disegnato

Alla lavagna avevamo disegnato la linea delle versioni con un ramo che si stacca e rientra. Ecco cosa racconta: il progetto va avanti su **main**, ogni tanto si pubblica una versione (le bandierine), e un ramo di lavoro si può staccare e poi riunire.



Da **v1.0** il progetto cresce; un ramo porta la **v1.1** e si riunisce; poi un grande cambiamento porta alla **v2.0**.

Regola d'oro: una versione già pubblicata **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi.

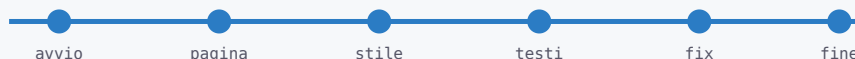
7. Casi d'uso: 15 grafi con spiegazione (fino a 5 livelli)

Vediamo Git/versioning "in azione". In ogni grafo il **pallino** è un commit/versione e la **linea** è il tempo (da sinistra a destra). I primi sono semplici; gli ultimi arrivano a **cinque livelli** di rami.

Come si legge un grafo: ogni **pallino** è un **commit** (una fotografia salvata). La **linea orizzontale** in alto è di solito **main**, il ramo principale. Le linee colorate che si staccano sono **branch** (rami di lavoro). Quando due linee si riuniscono è un **merge**. Le **bandierine gialle** sono le **release** (versioni pubblicate, es. v1.0).

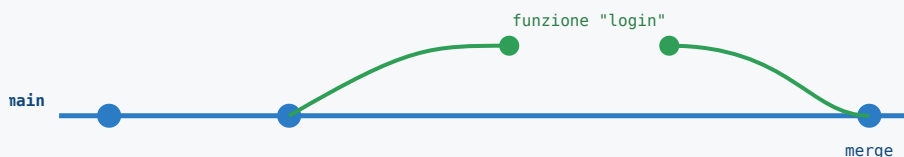
1. Sviluppo lineare (da soli, tutto dritto)

Il caso più semplice: un solo autore e nessun ramo. Ogni **commit** (una fotografia salvata del progetto, con un messaggio che dice cosa è cambiato) segue il precedente in fila, come le pagine di un diario scritte una dopo l'altra. La storia è una linea dritta: facilissima da leggere e da seguire. Va benissimo per progetti piccoli o quando lavori da solo. **Esempio:** costruisci la tua pagina web un pezzo alla volta — prima l'ossatura, poi lo stile, poi i testi — e ogni passo è un commit. Se qualcosa si rompe, torni indietro di un commit e riparti da lì senza aver perso nulla.



2. Una funzione su un branch, poi merge

Vuoi provare una funzione nuova senza rischiare di rompere quello che già funziona. Apri un **branch** (ramo): una strada di lavoro separata che parte dal **main**. Sul ramo sviluppi con calma, mentre il **main** resta sempre sano e usabile dagli altri. Quando la funzione è pronta e provata, la unisci al **main** con un **merge**. **Esempio:** nel tuo gioco aggiungi il "login" su un ramo apposta; finché non funziona bene, i compagni continuano a usare il gioco dal **main**. Il vantaggio è proprio questo: il lavoro in corso non disturba mai la versione buona.

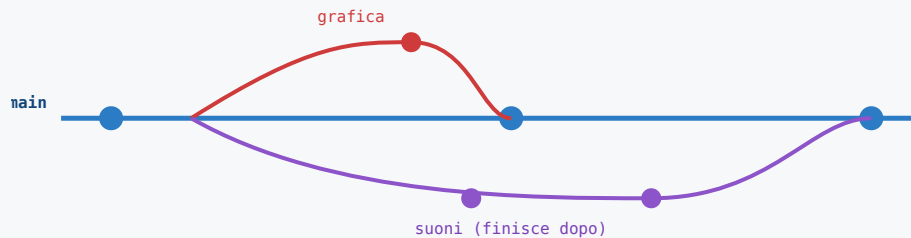


3. Due funzioni in parallelo, merge in tempi diversi

Due persone (o due idee) lavorano nello stesso periodo, ognuna sul suo ramo. Partono insieme ma non finiscono nello stesso momento: una funzione è pronta prima e si fonde nel main, l'altra ci mette di più e si fonde dopo. Il main raccoglie i pezzi **man mano** che sono pronti, senza aspettare che tutto sia finito.

Esempio: uno fa la "grafica" e la chiude in due giorni; un altro fa i "suoni" e li chiude a fine settimana.

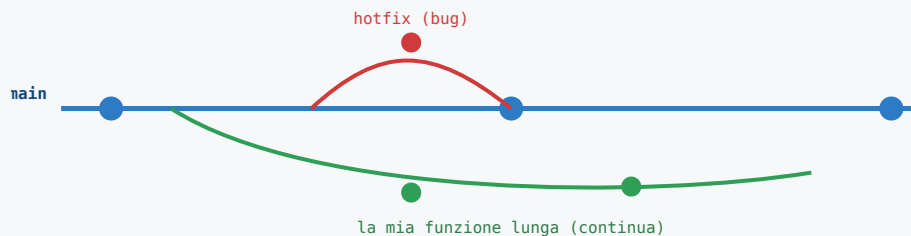
Così il progetto avanza a pezzi, senza che nessuno resti fermo ad aspettare gli altri.



4. Hotfix urgente durante una funzione lunga

Stai lavorando a una funzione lunga (giorni di lavoro), ma all'improvviso salta fuori un **bug grave** nella versione che usano gli altri (in "produzione"). Non puoi aspettare di finire la tua funzione. Apri un ramo **hotfix** (riparazione urgente) corto: parte dal main, corregge solo quel problema e viene subito riunito.

Poi torni tranquillo alla tua funzione, che nel frattempo è rimasta al sicuro sul suo ramo. **Esempio:** mentre costruisci un nuovo livello del gioco scopri che il punteggio non si salva più: hotfix veloce, e poi riprendi il livello da dove eri.



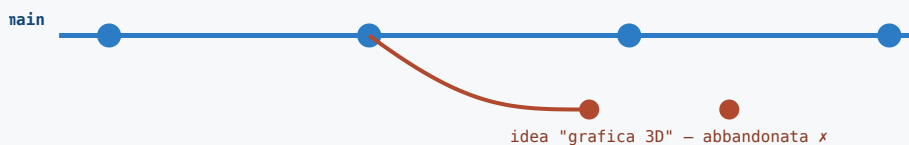
5. Le release pubblicate (bandierine sul main)

Man mano che il progetto migliora, ogni tanto si **pubblica** una versione: si mette una bandierina (un **tag**, cioè una **release**) sul main a segnare un punto stabile. Da `v1.0` (prima versione giocabile) si passa a `v1.1` (aggiungi i suoni), `v1.2` (aggiungi livelli), `v1.3` (correggi bug), fino a un cambiamento grosso che porta a `v2.0`. Ogni bandierina è una versione "congelata": da lì in poi non si tocca più; se cambi qualcosa fai una versione nuova. Serve per sapere sempre "quale versione ho in mano" e per poter tornare a una versione stabile in caso di problemi.



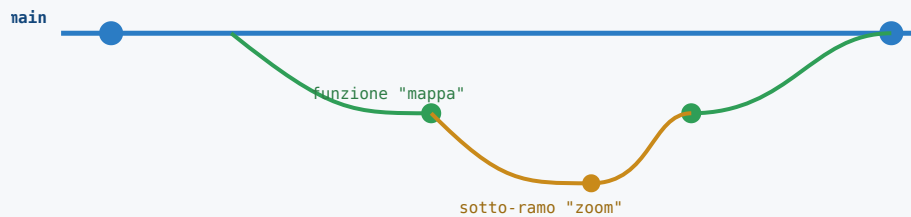
6. Un ramo abbandonato (idea scartata)

Non tutte le idee funzionano, ed è normalissimo. Qui un ramo viene provato ma **mai unito** al main: si prova la strada, si capisce che non va (troppo difficile, non piace, non serve) e la si lascia lì. Il bello di Git è che il main **non ne risente per niente**: la versione buona resta intatta, come se quell'esperimento non fosse mai esistito. **Esempio**: provi una "grafica 3D" su un ramo, ti accorgi che il gioco diventa lento e abbandoni il ramo. Nessun danno: solo una cosa imparata, senza aver rischiato il progetto vero.



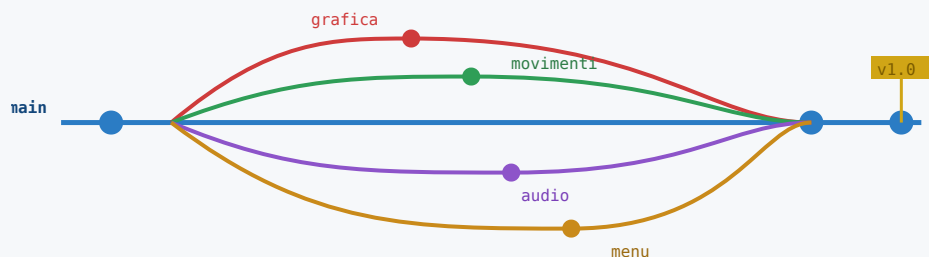
7. Un ramo che nasce da un altro ramo

A volte una funzione è così grande che, mentre la sviluppi sul suo ramo, ne stacchi un pezzo su un **sotto-ramo** (un ramo che nasce da un altro ramo). Si richiude dal più interno al più esterno: prima rientra il sotto-ramo dentro il ramo, poi il ramo dentro il main, come scatole una dentro l'altra. **Esempio:** la funzione "mappa" è grande; da lì stacchi il sotto-ramo "zoom della mappa"; finito lo zoom lo riunisci alla mappa, poi la mappa completa la riunisci al main. Serve per tenere ordinato anche il lavoro complesso, un pezzo alla volta.



8. Grande integrazione (progetto di gruppo)

Nel lavoro di squadra ognuno ha il suo ramo: chi la grafica, chi i movimenti, chi l'audio, chi il menu. Ognuno lavora in parallelo **senza pestare i piedi** agli altri, perché ogni ramo è separato. Alla fine tutti i rami confluiscono nel main, di solito con una **Pull Request** (la proposta di unire il proprio lavoro, che un compagno o il docente controlla prima di accettarla). Quando sono tutti dentro, nasce la versione completa v1.0. **Esempio:** il vostro progetto di classe, dove ognuno porta il suo pezzo e alla fine si monta il gioco intero. È esattamente il modo in cui si lavora nelle aziende.



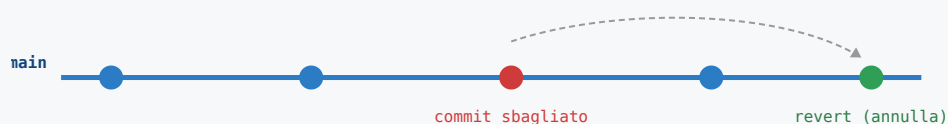
9. Ramo di manutenzione a lungo termine

Esci con la `v1.0` e il main va avanti verso la `v2.0` (che sarà tutta nuova). Ma c'è chi usa ancora la `v1` e ha bisogno di correzioni: non puoi obbligarlo a passare subito alla `v2`. Allora tieni un **ramo di manutenzione** parallelo, dove escono `v1.1`, `v1.2` con solo le correzioni (patch), senza le novità della `v2`. **Esempio:** un programma usato in azienda: alcuni restano sulla versione vecchia perché funziona, e tu continui a correggergliela mentre prepari quella nuova. Così accontenti sia chi vuole stabilità sia chi vuole novità.



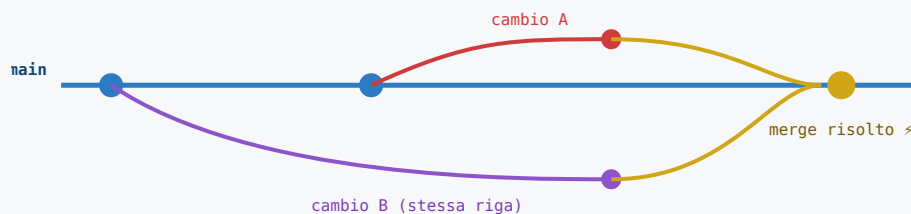
10. Revert: annullare una modifica sbagliata

Un commit ha introdotto un problema e vuoi tornare indietro. Regola d'oro: la **storia non si cancella** (serve a capire cosa è successo). Invece di cancellare, aggiungi un **nuovo commit** che annulla esattamente quello sbagliato: è il **revert**. Il risultato è come se la modifica sbagliata non ci fosse, ma nella storia resta traccia onesta sia dell'errore sia della sua correzione. **Esempio:** aggiungi una modifica che fa sparire il pulsante "salva"; te ne accorgi e fai revert: il pulsante torna, e chiunque guardi la storia capisce cosa era successo e come è stato risolto.



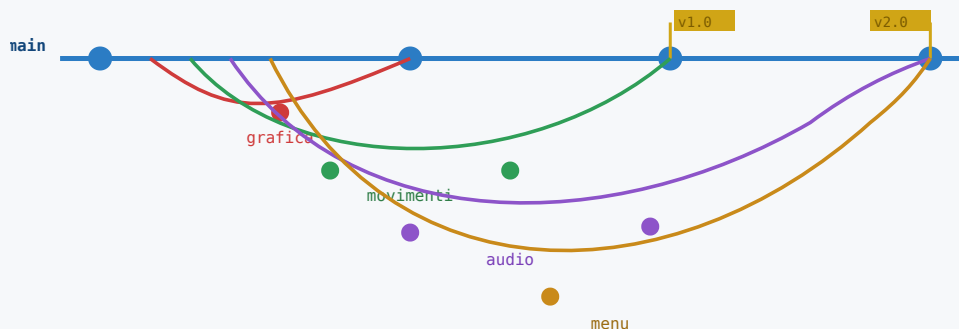
11. Conflitto di merge risolto

Due rami hanno cambiato **la stessa identica riga** in modo diverso. Quando li unisci, Git non può decidere da solo quale versione tenere: si ferma e segnala un **conflitto**. A quel punto una persona guarda le due versioni, sceglie cosa tenere (o le combina) e chiude l'unione con un commit speciale di "risoluzione". Non è un errore grave: è normale quando si lavora in tanti sullo stesso file. **Esempio:** due compagni cambiano il titolo della schermata iniziale in due modi diversi; al merge si decide insieme quale titolo tenere. Capire i conflitti è una delle cose più utili per il lavoro di gruppo.



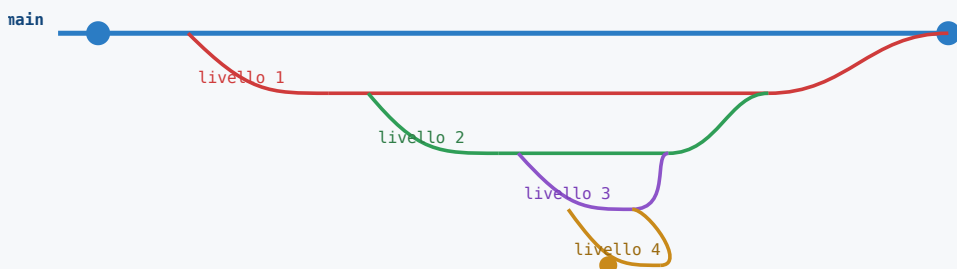
12. Squadra a 5 rami con due release

Un progetto vero di gruppo mette insieme più cose viste finora. Sul main si pubblicano le versioni (v1.0, poi v2.0), mentre **quattro rami** diversi — grafica, movimenti, audio, menu — lavorano in parallelo e rientrano in momenti diversi, non tutti insieme. Sono cinque linee vive contemporaneamente: una principale e quattro di lavoro. **Esempio:** la vostra squadra che porta il gioco dalla prima versione giocabile fino a una versione 2 molto più ricca, integrando i pezzi via via che sono pronti. E la fotografia realistica di come cresce un progetto di squadra.



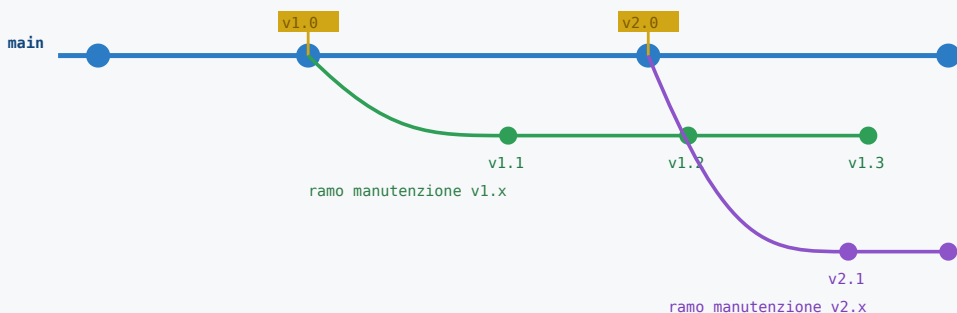
13. Rami annidati: cinque livelli di profondità

Una funzione grande si divide in pezzi sempre piu piccoli: dal main nasce un ramo (livello 1), da quel ramo un **sotto-ramo** (livello 2), e così via scendendo di **cinque livelli**. Ogni livello rientra nel precedente, dal piu interno al piu esterno, come le matrische. Serve per affrontare un lavoro complicato un pezzo alla volta, senza perdere il filo. **Esempio:** "nuovo mondo" contiene "mappa", che contiene "zoom", che contiene "mini-mappa", che contiene "icone". Ogni pezzo, finito, si richiude nel suo contenitore. E raro spingersi così in profondita, ma capire il meccanismo aiuta a non spaventarsi davanti ai progetti grandi.



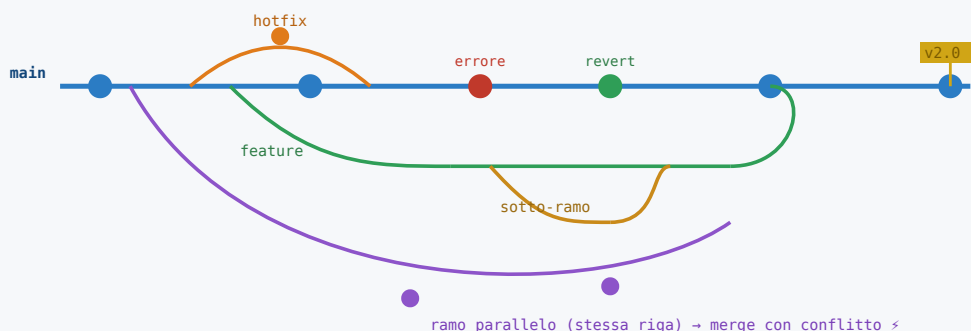
14. Due versioni major mantenute insieme

Dopo la `v1.0` il main punta alla `v2.0` (un cambiamento grosso), ma la `v1` resta in uso da qualcuno. Così tieni **due linee di manutenzione parallele**: il ramo `v1.x` sforna `v1.1` e `v1.2` (solo correzioni per chi è rimasto alla versione 1), e più tardi anche la `v2` avrà il suo ramo `v2.x`. Più versioni principali vive insieme, ognuna curata sul suo ramo. **Esempio:** un'app che mantiene sia la versione vecchia (per chi non vuole cambiare) sia quella nuova, correggendo i bug su entrambe. Le grandi aziende lo fanno sempre: è il segno di un prodotto usato sul serio.



15. Storia realistica: rami, hotfix, revert e conflitto

Il quadro completo, con tutto insieme: il main avanza e pubblica le release; una **feature** lunga procede sul suo ramo e ne stacca un **sotto-ramo**; salta fuori un **hotfix** urgente da riunire subito; un **revert** annulla un errore lasciandone traccia; e un ramo parallelo che ha toccato la stessa riga genera un **merge con conflitto** da risolvere. Cinque tipi di attività che convivono, esattamente come in un progetto vero. **Esempio:** una settimana tipo del vostro gioco di classe, dove capitano un po' tutte queste cose. Se sai leggere questo grafo, sai leggere quasi qualsiasi storia di progetto.



In sintesi: con pochi elementi — commit, branch, merge, tag/release — si descrivono **tutte** le situazioni di un progetto reale, dalla più semplice (linea dritta) alle più complesse (integrazione di squadra, manutenzione parallela, conflitti). Saper **leggere** questi grafi è già metà del lavoro.

8. Riepilogo (la lavagna in tre righe)

1. **GitHub** è un'**azienda** e un **servizio** online dove tieni i tuoi **repository di codice** (con Git sotto, come motore).
2. Come servizio è una **piattaforma pronta (PaaS)**, della famiglia **...aaS**, e si paga in **Freemium** (base gratis, extra a pagamento).
3. Il progetto cresce per **versioni** (**1.2.3**): major, minor, patch — e una versione pubblicata non si tocca più.

Per vedere tante situazioni diverse con i grafi, guarda la scheda "Versioning — 11 casi d'uso".

9. Dal magazzino agli strumenti dell'IA

Fin qui abbiamo visto **dove** vive e cresce il lavoro: Git, GitHub e le versioni. Adesso vediamo **due strumenti di Intelligenza Artificiale** che usiamo insieme a quel materiale: **Gemini** (a cui diamo un file .md per farci spiegare) e **NotebookLM** (a cui diamo un PDF per ripassare).

10. I file Markdown (.md)

1. Un file **Markdown** (estensione **.md**) e **testo puro** con pochi simboli per la formattazione:
2. **#** a inizio riga fa un **titolo**;
3. ****parola**** mette in **grassetto**;
4. una riga con **1.** fa un elenco numerato.
5. Si legge ovunque (Blocco note, o su GitHub nel browser) e non contiene immagini pesanti: solo contenuto.
6. Per questo è perfetto per l'**IA** (Intelligenza Artificiale): la legge molto bene e ci lavora senza confondersi. Nel corso diamo le dispense anche in .md proprio per poterle dare a Gemini.

Cosa devi saper spiegare: perché un .md si dice "testo puro" e perché è comodo con l'IA.

11. Gemini: l'IA generativa, e come usarla con un file .md

1. **Gemini** è un'IA **generativa**: le scrivi una richiesta (un "prompt", cioè un comando testuale) e lei genera una risposta. Può spiegare, tradurre, riassumere, fare domande di ripasso.
2. Attenzione: l'IA può anche **sbagliare o inventare** (si chiamano "allucinazioni"): quello che dice va sempre controllato, soprattutto date, numeri e fonti.

12.1 Passi per usare Gemini con un file .md

1. Nel browser vai su questo indirizzo, con l'account della scuola:

`gemini.google.com`

2. Apri il file **.md** (col Blocco note o su GitHub), **seleziona tutto e copialo**.
3. Torna su Gemini e **incolla** il testo nella casella del messaggio (o allega il file, se puoi).
4. Scrivi la richiesta. Per esempio:

`Spiegami con parole semplici la parte che non capisco: [incolla la frase].`

11.2 Quale versione di Gemini scegliere

1. In alto Gemini ha un menu per la **versione del modello**. I nomi/numeri cambiano; conta a cosa serve ciascuna:
2. **Flash** (veloce): risposte rapide per cose semplici — spiegazioni, traduzioni, riassunti brevi. È la scelta di tutti i giorni.
3. **Pro** (per esempio "3.1 Pro", più potente): per compiti complessi e testi lunghi. Più lenta ma più capace.

4. **Ragionamento** (per esempio "3.6 ragionamento", che "pensa" prima di rispondere): per problemi difficili, passo per passo. La piu lenta ma la piu accurata.
5. Regola pratica: parti sempre da **Flash**; se la risposta e complessa o ti sembra sbagliata, passa a **Pro** o **Ragionamento** e richiedi.
6. Guarda a **cosa serve** la versione (veloce / potente / ragiona), non solo al numero.

Le versioni piu potenti o di "ragionamento", con l'account gratuito, possono avere un limite giornaliero: usale quando servono davvero, per il resto va bene Flash.

12. NotebookLM: il tutor sulle fonti, e come usarlo con un PDF

1. **NotebookLM** e diverso da Gemini: risponde **solo** sui documenti che carichi. Se carichi un PDF, ti risponde usando **quel** PDF e non inventa fuori da li.
2. Per questo e ottimo per **studiare e ripassare** su una fonte precisa (una dispensa, un capitolo): resta fedele al testo.

12.1 Passi per usare NotebookLM con un PDF

1. Nel browser vai su questo indirizzo, con l'account della scuola:

`notebooklm.google.com`

2. Clicca **Crea nuovo**, poi **Carica**, e scegli un file **PDF**.
3. Aspetta che la fonte compaia a sinistra: e pronta.
4. Nel riquadro della chat scrivi, per esempio:

`Fammi un riassunto semplice di questo documento.`

Se non hai un PDF sul computer: su github.com entra nella cartella del documento, clicca sul PDF e poi sull'icona di download (freccia in giu) in alto a destra; lo trovi nella cartella Download.

13. Il flusso completo: i tre ambienti insieme

1. Il materiale del corso sta su **Git/GitHub** (versionato).
2. Da li si **scarica** il PDF (per leggere/stampare) o il file **.md** (per l'IA).
3. Il **.md** si da a **Gemini** per farsi spiegare o tradurre le parti difficili.
4. Il **PDF** si carica su **NotebookLM** per interrogarlo e ripassare restando sulla fonte.
5. Il lavoro finito si **consegna su Classroom**.

Strumento	A cosa serve	Cosa gli dai
Git / GitHub	Conservare e condividere il lavoro versionato	I file del progetto
Gemini	Spiegare, tradurre, riassumere	Un testo o un file .md
NotebookLM	Ripassare restando fedele alla fonte	Un documento (per esempio un PDF)

14. Uso responsabile dell'IA

1. **Capire, non copiare:** l'IA è un aiuto per imparare, non un modo per saltare il lavoro. Vale la prova del nove: saperlo spiegare a parole proprie.
2. **Controllare sempre:** l'IA può sbagliare o inventare; verifica quello che dice, soprattutto date, numeri e fonti.
3. **Privacy e dati:** non inserire dati personali, di compagni o di famiglia negli assistenti.

Prendi appunti a mano: prima di usare gli strumenti, disegna sul quaderno tre riquadri (Git, Gemini, NotebookLM) e, per ognuno, a cosa serve e cosa gli dai.

15. Glossario

Parola	Vuol dire
GitHub	il sito-servizio (azienda di Microsoft) che ospita i progetti online
Git	lo strumento (gratis) che tiene la storia di un progetto — il "motore" sotto GitHub
repository	la cartella-progetto con dentro i file e tutta la loro storia
commit	un salvataggio (una fotografia) del progetto con un messaggio
branch	una strada di lavoro parallela, per provare senza rovinare il main
merge	unire un ramo con quello principale (main)
PaaS	"piattaforma come servizio": una piattaforma online già pronta (es. GitHub)
IaaS / SaaS	gli altri servizi ...aaS: infrastruttura (server nudi) / software pronto nel browser
Freemium	base gratis, funzioni avanzate a pagamento
versioning	dare un numero (v1.2.3) alle versioni: major . minor . patch
release	una versione pubblicata e "congelata"; non si riusa mai un numero già dato

Markdown (.md)	file di testo puro con pochi simboli (# titoli, ** grassetto): ottimo per l'IA
IA generativa	intelligenza artificiale che genera testo su richiesta (per esempio Gemini)
prompt	la richiesta scritta che dai all'IA
allucinazione	quando l'IA inventa una risposta sbagliata: va sempre controllata
Gemini	l'IA generativa di Google che usiamo a scuola (versioni Flash / Pro / ragionamento)
NotebookLM	tutor che risponde solo sui documenti caricati (per esempio un PDF)
Classroom	dove il docente dà i compiti e raccoglie i lavori